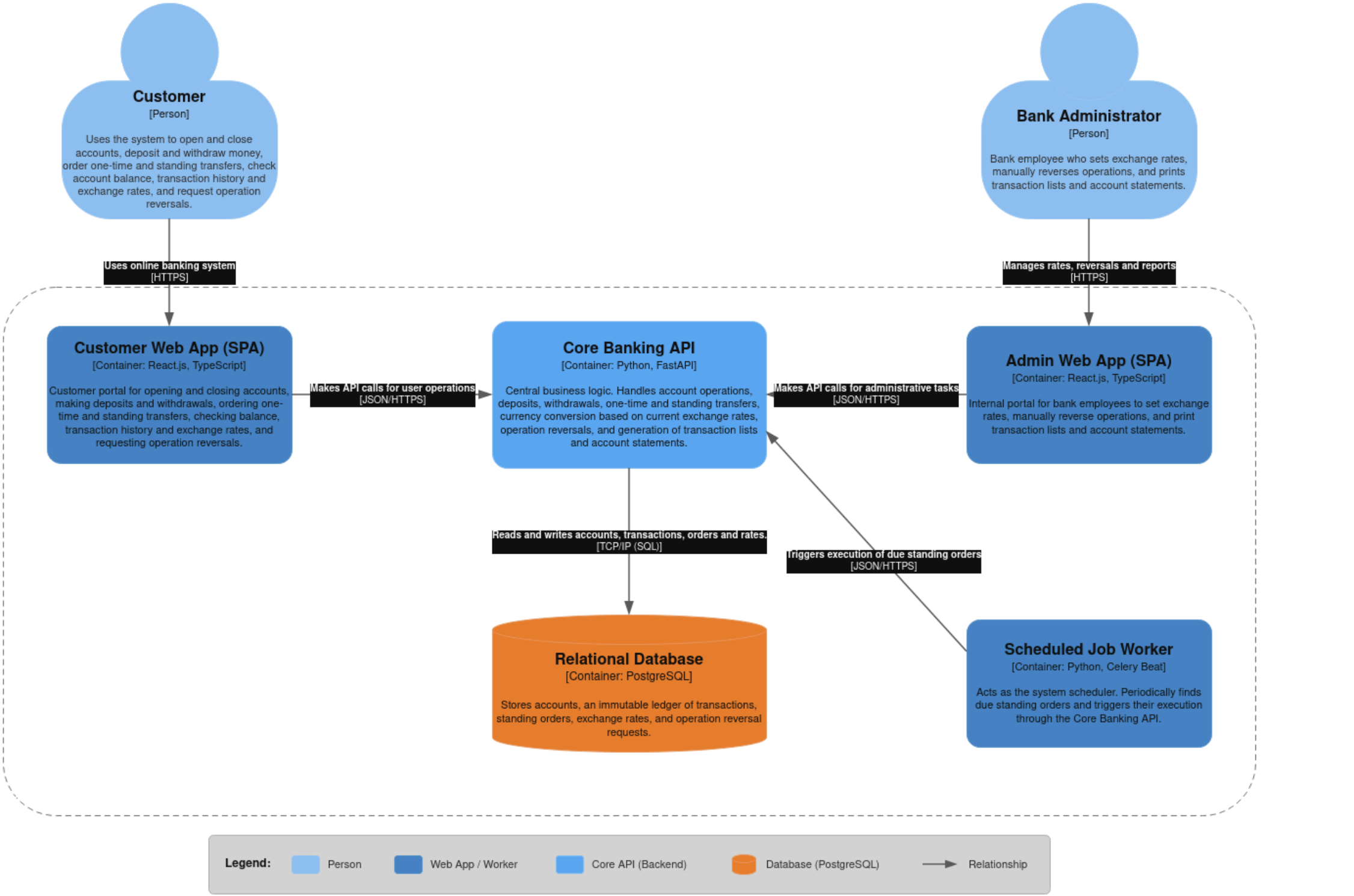
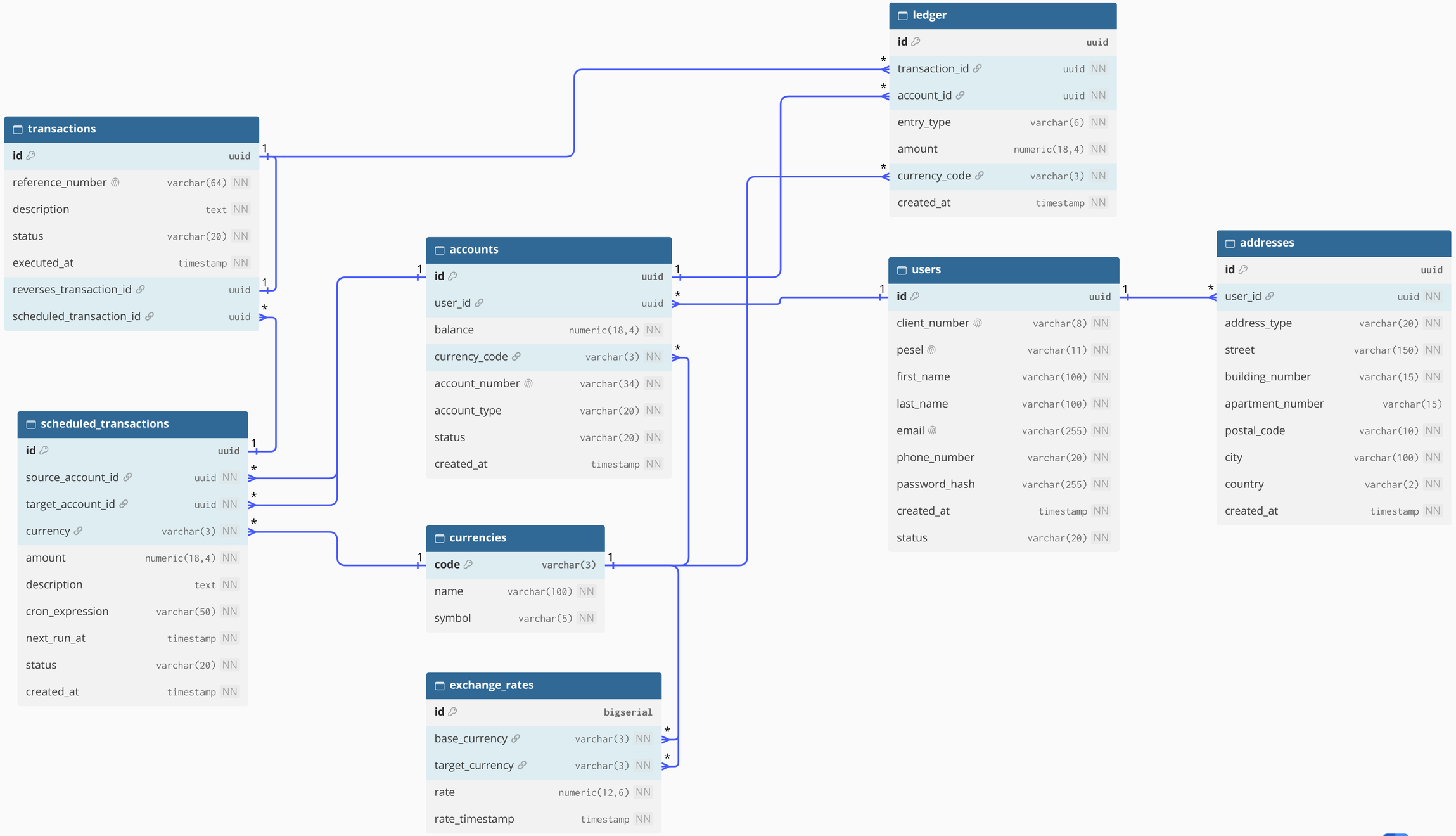
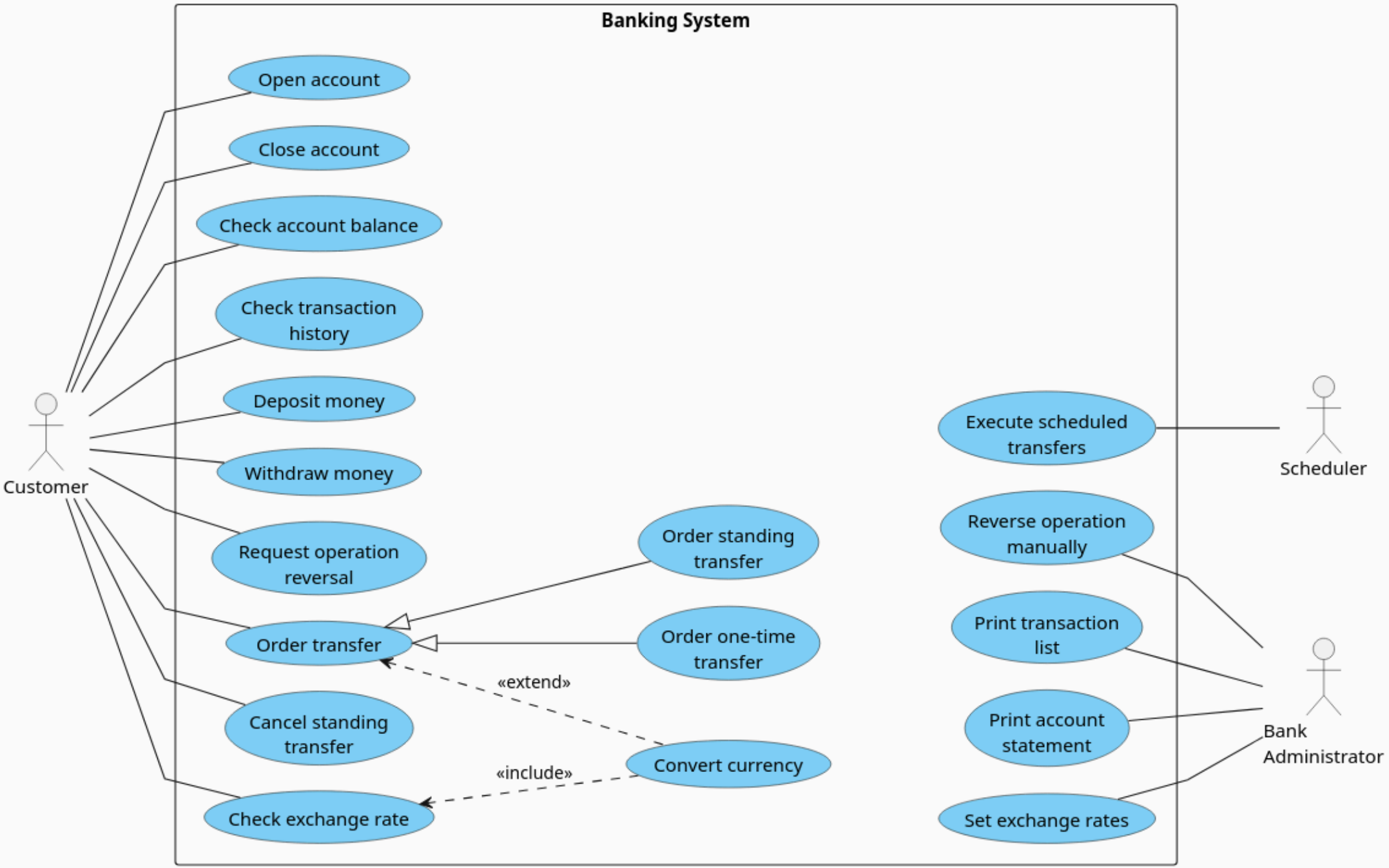

Customer
[Person]
Uses the system to open and close accounts, deposit and withdraw money, order one-time and standing transfers, check account balance, transaction history and exchange rates, and request operation reversals.
Bank Administrator
[Person]
Bank employee who sets exchange rates, manually reverses operations, and prints transaction lists and account statements.
Uses online banking system
[HTTPS]
Manages rates, reversals and reports
[HTTPS]
Customer Web App (SPA)
[Container: React.js, TypeScript]
Customer portal for opening and closing accounts, making deposits and withdrawals, ordering one-time and standing transfers, checking balance, transaction history and exchange rates, and requesting operation reversals.
Makes API calls for user operations
[JSON/HTTPS]
Core Banking API
[Container: Python, FastAPI]
Central business logic. Handles account operations, deposits, withdrawals, one-time and standing transfers, currency conversion based on current exchange rates, operation reversals, and generation of transaction lists and account statements.
Makes API calls for administrative tasks
[JSON/HTTPS]
Admin Web App (SPA)
[Container: React.js, TypeScript]
Internal portal for bank employees to set exchange rates, manually reverse operations, and print transaction lists and account statements.
Reads and writes accounts, transactions, orders and rates.
[TCP/IP (SQL)]
Triggers execution of due standing orders
[JSON/HTTPS]
Relational Database
[Container: PostgreSQL]
Stores accounts, an immutable ledger of transactions, standing orders, exchange rates, and operation reversal requests.
Scheduled Job Worker
[Container: Python, Celery Beat]
Acts as the system scheduler. Periodically finds due standing orders and triggers their execution through the Core Banking API.
Legend:
Person
Web App / Worker
Core API (Backend)
Database (PostgreSQL)
Relationship



dbdiagram.io


Banking System
Open account
Close account
Check account balance
Check transaction history
Deposit money
Withdraw money
Request operation reversal
Order transfer
Cancel standing transfer
Check exchange rate
Order standing transfer
Order one-time transfer
Convert currency
«extend»
«include»
Execute scheduled transfers
Reverse operation manually
Print transaction list
Print account statement
Set exchange rates
Customer
Scheduler
Bank Administrator

Mini-Bank

John Doe | Logout

Dashboard | Transfers | Scheduled Transfers

zł PLN

Checking account (PLN)

PL12 3456 7890 1234 5678 9012 3456

Funds available

12 450,00 zł

Make a Transfer

€ EUR

Foreign currency (EUR)

PL98 7654 3210 9876 5432 1098 7654

Funds available

1 200,50 €

Make a Transfer

\+

Open a new account

Recent operations

Executed at	Description	Account	Type	Amount
15.10.2026 09:42	Groceries	Checking account (PLN) •••• 3456	DEBIT	−189,90 zł
14.10.2026 16:18	Hotel reservation	Foreign currency (EUR) •••• 7654	DEBIT	−85,00 €
14.10.2026 08:30	Salary	Checking account (PLN) •••• 3456	CREDIT	+8 500,00 zł
13.10.2026 11:06	Refund	Foreign currency (EUR) •••• 7654	CREDIT	+50,00 €
12.10.2026 18:25	Internet bills	Checking account (PLN) •••• 3456	DEBIT	−120,00 zł

Show More

Mini-Bank

John Doe

Logout

Dashboard | Transfers | Scheduled Transfers

New Transfer

Order one-time transfer and review currency conversion before confirmation.

Source Account

Checking account (PLN) — PL12 3456 ... (Available: 12 450,00 zł)

Recipient IBAN / Account Number

PL00 0000 0000 0000 0000 0000 0000

Amount

100.00

Currency

EUR

€ **Currency Conversion**

Current rate

1 EUR = 4.312000 PLN

2026-10-20 12:00:00

Estimated total charge

431.20 PLN

Fee

0.00 PLN

Description

VAT invoice 123/2026

Make a Transfer

Active Standing Orders

Scheduled transactions executed by background tasks

Recipient Account	Amount	Schedule	Next Execution	Status	Actions
PL88 1234 ... 9999	**1 500,00 zł**	Monthly (1st day)	01.11.2026	ACTIVE	Cancel
PL12 3456 ... 5678	**250,00 zł**	Every Week (Monday)	26.10.2026	ACTIVE	Cancel
PL98 7654 ... 5432	**85,00 €**	Daily	21.10.2026	ACTIVE	Cancel

Create New Standing Order

Set up a recurring transfer with a selected schedule

Source Account

PL12 3456 7890 1234 5678 9012 3456

Destination Account

PL88 1234 5678 9012 3456 7890 9999

Amount

1 500,00

Currency

EUR

Description

Rent payment

Frequency

Every Month (1st day)

Start Date

01.11.2026

Set Standing Order

Exchange Rates Board

Pair	Current Rate	Last Updated	Action
EUR / PLN	**4.312000**	2026-10-20 09:00	Edit Rate
USD / PLN	**3.985000**	2026-10-20 09:00	Edit Rate
GBP / PLN	**5.189000**	2026-10-20 09:00	Edit Rate

Operation Reversal

Search by reference number and manually reverse posted transactions

Search by Reference Number

TRX-12345678-2026

Reverse Transaction

Date	From Account	To Account	Amount	Status
2026-10-20	PL12 3456 7890 1234 5678 9012 3456	PL88 1234 5678 9012 3456 7890 9999	**1 500,00 zł**	POSTED

TECH STACK

Backend - Python (FastAPI and SQLAlchemy probably???)

Industry context

- Java Spring Boot and C# .NET are the dominant technologies in the banking sector (Spring Boot has been widely used in core banking systems for decades given its support for transactions `@Transactional(isolation = ...)` and pessimistic locks `@Lock(LockModeType.PESSIMISTIC_WRITE)`)
- Given the project's basic level of complexity, our proficiency in Python language and lack of experience in Java and C#, we decided to prepare a demonstration version using Python.

Justification

- The team possesses a decent level of proficiency in Python. Choosing this environment minimizes the risk of implementation errors and allows to focus on domain logic (double-entry bookkeeping, ledger).
- Through integration with Pydantic and the standard `Decimal` type, the backend ensures strict validation of incoming JSON payloads at the system boundaries, eliminating the risk of floating-point rounding errors in financial operations.

Database - PostgreSQL 16 or PostgreSQL 17

Justification

- Native support for arbitrary-precision numeric types—specifically `numeric(18, 4)` for monetary amounts and `numeric(12, 6)` for exchange rates—eliminates currency rounding errors.
- PostgreSQL offers advanced concurrency control mechanisms (MVCC) and row-level locking (SELECT ... FOR UPDATE) for handling concurrent requests that affect an account.
- The PostgreSQL relational database is the standard for transactional and accounting systems. It guarantees the atomicity of multi-stage banking operations (e.g. the simultaneous creation of a transaction record and balanced entries in the general ledger).
- Defined foreign keys, UNIQUE and CHECK constraints ensure data consistency at the database engine level.

Frontend - React.js + TypeScript (SPA)

Justification

- Single Page Application (SPA) architecture ensures the separation of the presentation layer from the application's business logic. The frontend layer communicates with the backend exclusively through a defined REST API, utilizing the JSON data format.
- React library enables building reusable components (account cards, forms for validating IBAN formats, transaction history tables).
- Static typing in TypeScript ensures contracts for data structures transmitted from the backend (using interfaces that correspond to Pydantic or database models); it eliminates errors at the client–server interface.
- Taking into account the complexity of the project and its focus on architecture design, the frontend primarily serves to demonstrate how the architecture functions, which does not require a high level of complexity in the visual layer (e.g. using complex frameworks such as Angular).